Group Members :	1. Aldwin Leonard Wijaya - 2902562183 2. Melvin Jusuf - 2902681714 3. Yudith Cory Rahargo - 2902574504

AOL GRADING COMPONENTS TABLE

This page serves as an extended 'Table of Contents' for your AoL Submissions.
Please fill this page carefully, as the table will serve as the baseline for your AoL Grading.
Fill in all yellow-colored fields.

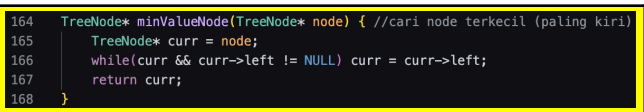
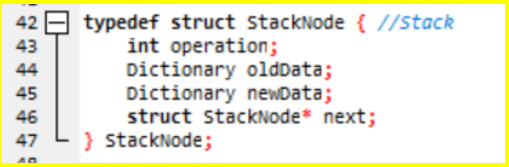
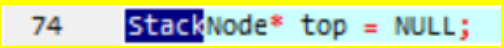
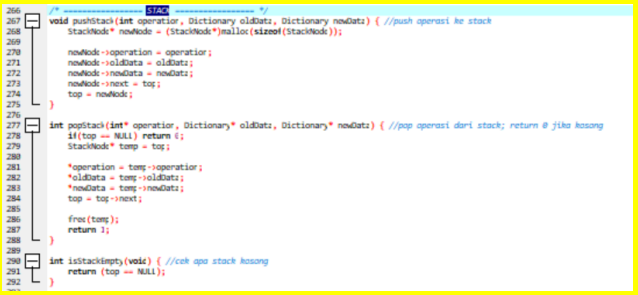
No	LO-LObj-SO	Rubric Assessment Criteria	Question	Excellent Requirement (Score 85-100)
1	LO3-LObj 2.2 -SO 2	3.1 Ability to construct at least 6 kinds of data structures using C (70%)	The project consists of data structure and well construct to solve the problem [40 Points]	Able to construct at least 6 kinds of data structures using C **[Total 70 %]**
2			Student able to describe how to use the data [30 Points]	
3	LO3-LObj 2.2 -SO 2	3.2 Ability to manipulate at least 6 kinds of data structures using C (30%)	Show that the Application able to run well [30 Points]	Able to solve 76- 100% the problem with fully functional feature **[Total 30 %]**

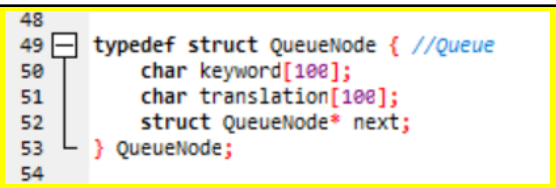
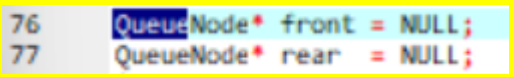
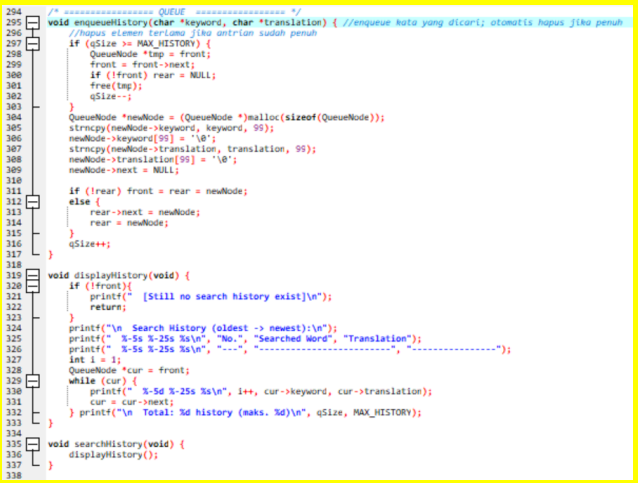
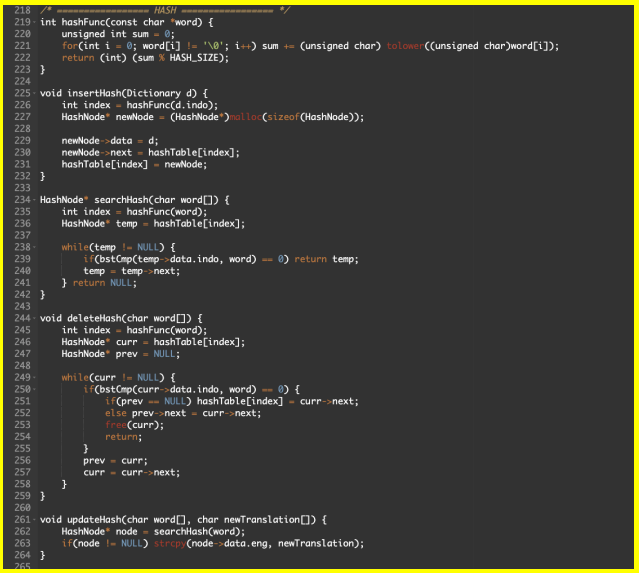
Assessment 1: Able to construct at least 6 kinds of data structures using C [70 %]

No.	Implemented Data Structure	Program Lines	Documentation Pages
....	*Arrays / Linked lists / Queues / Stacks / Binary trees / Hash tables / ....*	*List all the lines that you want to register as the data structure* *Ex: 58 - 63* 58 `int find_a(int parent[], int x)` 59 `{` 60 `while (parent[x] != x)` 61 `x = parent[x];` 62 `return x;` 63 `}` *Screenshot is **NOT** necessary; this is just an example.*	*List where this functionality explain in your documentation* *Page 4-5*
1	Array	20 `typedef struct { //Array` 21 `char indo[100];` 22 `char eng[100];` 23 `} Dictionary;`	Halaman 10 - 11 Deskripsi: Menu 5: *Display All*

| | | ```
Dictionary dictArray[MAX];
int count = 0;
``` | *(Array)*. Menampilkan seluruh kata di dalam memori/array secara berurutan sesuai input awal pengguna. |
| 2 | DCLL | ```
typedef struct Node { //DCLL
    Dictionary data;
    struct Node* next;
    struct Node* prev;
} Node;
```<br>```
void insertDCLL(Dictionary d) {  //sisipkan di akhir DCLL
    Node* newNode = (Node*)malloc(sizeof(Node));
    newNode->data = d;

    if(head == NULL) {
        head = newNode;
        newNode->next = newNode;
        newNode->prev = newNode;
    } else {
        Node* tail = head->prev;
        tail->next = newNode;
        newNode->prev = tail;
        newNode->next = head;
        head->prev = newNode;
    }
}
```<br>```
void deleteDCLL(char word[]) {
    if(head == NULL) return;
    Node* curr = head;
    do {
        if(strcmp(curr->data.indo, word) == 0) {
            if(curr->next == curr) {
                free(curr);
                head = NULL;
                return;
            }
            curr->prev->next = curr->next;
            curr->next->prev = curr->prev;
            if(curr == head) head = curr->next;
            free(curr);
            return;
        }
        curr = curr->next;
    } while(curr != head);
}
``` | Halaman 2, 4, 5 - 6<br><br>Deskripsi: *Struktur Latar Belakang* untuk Operasi CRUD: Menu 1 (*Add Word*), Menu 2 (*Delete Word*), dan Menu 3 (*Change Translate Word*). |
| 3 | BST | ```
typedef struct TreeNode { //BST
    Dictionary data;
    struct TreeNode* left;
    struct TreeNode* right;
} TreeNode;
```<br>```
int bstCmp(const char *a, const char *b) { //perbandingan case-insensitive untuk BST
    char la[100], lb[100];
    toLowerStr(a, la, sizeof(la));
    toLowerStr(b, lb, sizeof(lb));
    return strcmp(la, lb);
}
``` | Halaman 11 - 12<br><br>Deskripsi: Menu 6: *Display Sorted (BST)*. Menampilkan kata yang |

		   	sudah tersusun rapi secara alfabetis (A-Z) menggunakan teknik penelusuran *In-Order traversal*.
4	Stack	  	Halaman 13 - 14 Deskripsi: Menu 7: *Undo Last Operation (Stack)*. Digunakan untuk membatalkan aksi modifikasi terakhir dengan prinsip LIFO (*Last In, First Out*).

5	Queue	  	Halaman 15 - 16 Deskripsi: Menu 8: *Word Search History*. Menampilkan riwayat pencarian kata terakhir Anda secara berurutan menggunakan prinsip FIFO (*First In, First Out*).
6	Hash	struct:  function: 	Halaman 7 - 9 Deskripsi: Menu 4: *Search Word (Hash + Trie)*. Digunakan saat ingin mencari kata secara langsung (*direct hit*) tanpa penelusuran satu per satu.
7	Trie	struct:  function:	Halaman 7 - 9, 17 - 18 Deskripsi: Menu 4: *Search Word* dan Menu 9: *Prefix Search*. Memfasilitasi pencarian

		``c
/* ================ TRIE ================ */
TrieNode* createTrieNode() {
    TrieNode* newNode = (TrieNode*)malloc(sizeof(TrieNode));
    newNode->endWord = 0;
    newNode->fullWord[0] = '\0';
    for(int i = 0; i < TRIE_ALPHA; i++) newNode->child[i] = NULL;
    return newNode;
}

void insertTrie(const char *word) {
    if (!rootTrie) rootTrie = createTrieNode();
    TrieNode *cur = rootTrie;

    for (int i = 0; word[i]; i++) {
        int idx = trieIndex(word[i]);

        if (idx < 0) continue;
        if (!cur->child[idx]) cur->child[idx] = createTrieNode();
        cur = cur->child[idx];
    }
    cur->endWord = 1;
    strncpy(cur->fullWord, word, 99);
    cur->fullWord[99] = '\0';
}

int deleteTrieHelper(TrieNode* curr, char word[], int depth) {
    if(curr == NULL) return 0;
    if(word[depth] == '\0') {
        curr->endWord = 0;
        return 1;
    }
    int index = tolower(word[depth]) - 'a';

    if(index < 0 || index > 25) return 0;
    return deleteTrieHelper(curr->child[index], word, depth + 1);
}

void deleteTrie(char word[]) {
    deleteTrieHelper(rootTrie, word, 0);
}
```

```c

TrieNode *searchPrefix(const char *prefix) { //cari node akhir dari prefix (case-insensitive)
    if (!rootTrie) return NULL;
    TrieNode *cur = rootTrie;
    for (int i = 0; prefix[i]; i++) {
        int idx = trieIndex(tolower((unsigned char)prefix[i]));
        if (idx < 0 || !cur->child[idx]) return NULL;
        cur = cur->child[idx];
    }
    return cur;
}

void collectWords(TrieNode *node) {
    if(node->endWord && suggestCount < MAX_SUGGEST)
        strcpy(suggestList[suggestCount++], node->fullWord);
    for(int i = 0; i < TRIE_ALPHA; i++)
        if(node->child[i]) collectWords(node->child[i]);
}


```

berbasis awalan (*prefix*) untuk menghasilkan fitur *auto-complete* atau memberikan rekomendasi kata

| 8 | Heap | struct:<br>```c
char heap[10][100];
int heapSize = 0;
int loadedWords = 0;
```<br>function:<br>```c
/* ================ HEAP ================ */
void insertHeap(char word[]) { //insert suggestion
    if(heapSize >= 10) return;
    strcpy(heap[heapSize], word);
    heapSize++;
}

void traversalTrie(TrieNode* curr, char word[], int level) {
    if(curr->endWord) {
        word[level] = '\0';
        insertHeap(word);
    }

    for(int i = 0; i < 26; i++) {
        if(curr->child[i] != NULL) {
            word[level] = i + 'a';

            traversalTrie(curr->child[i], word, level + 1);
        }
    }}

void prefixSearch() {
    char prefix[100];
    printf("Input Prefix: ");
    scanf("%s", prefix);
    clearInputBuffer();

    TrieNode* curr = rootTrie;
    for(int i = 0; prefix[i] != '\0'; i++) {

    int index = tolower((unsigned char)prefix[i]) - 'a';
    if(index < 0 || index > 25){
        printf("Invalid prefix.\n");
        return;
    }

    if(curr->child[index] == NULL) {
            printf("No matching words.\n");
            return;
        }
        curr = curr->child[index];
    }

    heapSize = 0;
    char temp[100];
    strcpy(temp, prefix);
    traversalTrie(curr, temp, strlen(prefix));

    printf("Suggestions:\n");
    for(int i = 0; i < heapSize; i++) printf("- %s\n", heap[i]);
}
```<br>Deskripsi: Menu 9: *Prefix Search*. Bekerja dengan insertHeap() untuk menyimpan sementara hasil *Depth-First Search* (DFS) dari Trie ke dalam array heap, dengan batasan maksimal 10 kata rekomendasi. | Halaman 17 - 18 |
```

9	*File Processing*		Halaman 1, 19 - 20 Deskripsi: Menu 0: *Save & Exit* dan *Loading* awal program. Menangani pemuatan data kamus saat *compile* awal. Memastikan persistensi data dengan fungsi saveFile() agar modifikasi RAM tersimpan di dictionary.txt.

Assessment 2: Show that the Application able to run well **[30 %]**
A project with rigorous documentation CAN opt NOT to submit a live demo.

Recording Links of the project demo:
Screenshots documentation

You may put "NO DEMO" instead if no live demo is provided.